Mini RFC: Transport Parameter

April 25, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

1 Overview

During connection establishment, both endpoints make authenticated declarations of their transport parameters. Endpoints are required to comply with the restrictions that each parameter defines; the description of each parameter includes rules for its handling.

Transport parameters are declarations that are made unilaterally by each endpoint. Each endpoint can choose values for transport parameters independent of the values chosen by its peer.

QUIC includes the encoded transport parameters in the cryptographic handshake. Once the handshake completes, the transport parameters declared by the peer are available. Each endpoint validates the values provided by its peer.

An endpoint MUST treat receipt of a transport parameter with an invalid value as a connection error of type `TRANSPORT_PARAMETER_ERROR`.

An endpoint MUST NOT send a parameter more than once in a given transport parameters extension. An endpoint SHOULD treat receipt of duplicate transport parameters as a connection error of type `TRANSPORT_PARAMETER_ERROR`.

Endpoints use transport parameters to authenticate the negotiation of connection IDs during the handshake.

ALPN allows clients to offer multiple application protocols during connection establishment. The transport parameters that a client includes during the handshake apply to all application protocols that the client offers. Application protocols can recommend values for transport parameters, such as the initial flow control limits. However, application protocols that set

constraints on values for transport parameters could make it impossible for a client to offer multiple application protocols if these constraints conflict.

2 QUIC Transport Parameters Extension

QUIC transport parameters are carried in a TLS extension. Different versions of QUIC might define a different method for negotiating transport configuration.

Read Section 4.2 of RFC 8446 to learn TLS Extensions.

Including transport parameters in the TLS handshake provides integrity protection for these values.

```
enum {
    quic_transport_parameters(0x39), (65535)
} ExtensionType;
```

The extension_data field of the quic_transport_parameters extension contains a value that is defined by the version of QUIC that is in use.

The quic_transport_parameters extension is carried in the ClientHello and the EncryptedExtensions messages during the handshake. Endpoints MUST send the quic_transport_parameters extension; endpoints that receive ClientHello or EncryptedExtensions messages without the quic_transport_parameters extension MUST close the connection with an error of type `0x016d` (equivalent to a fatal TLS missing_extension alert).

Transport parameters become available prior to the completion of the handshake. A server might use these values earlier than handshake completion. However, the value of transport parameters is not authenticated until the handshake completes, so any use of these parameters cannot depend on their authenticity. Any tampering with transport parameters will cause the handshake to fail.

3 Transport Parameter Encoding

The extension_data field of the quic_transport_parameters extension defined above contains the QUIC transport parameters. They are encoded as a sequence of transport parameters, as shown in Figure 1:

```
Transport Parameters {
    Transport Parameter (..) ...,
}
```

Figure 1: *Sequence of Transport Parameters*

Each transport parameter is encoded as an (identifier, length, value) tuple, as shown in Figure 2:

The Transport Parameter Length field contains the length of the Transport Parameter Value field in bytes.

```
Transport Parameter {
    Transport Parameter ID (i),
    Transport Parameter Length (i),
    Transport Parameter Value (..),
}
```

Figure 2: *Sequence of Transport Parameters*

QUIC encodes transport parameters into a sequence of bytes, which is then included in the cryptographic handshake.

3.1 Reserved Transport Parameters

Transport parameters with an identifier of the form 31 * N + 27 for integer values of N are reserved to exercise the requirement that unknown transport parameters be ignored. These transport parameters have no semantics and can carry arbitrary values.

3.2 Transport Parameter Definitions

This section details the transport parameters defined in this document.

Many transport parameters listed here have integer values. Those transport parameters that are identified as integers use a variable-length integer encoding. Transport parameters have a default value of 0 if the transport parameter is absent, unless otherwise stated.

The following transport parameters are defined:

- **original_destination_connection_id (`0x00`)**: This parameter is the value of the Destination Connection ID field from the first `Initial` packet sent by the client. This transport parameter is only sent by a server.
- **max_idle_timeout (`0x01`)**: The maximum idle timeout is a value in milliseconds that is encoded as an integer. Idle timeout is disabled when both endpoints omit this transport parameter or specify a value of 0.
- **stateless_reset_token (`0x02`)**: A stateless reset token is used in verifying a stateless reset. This parameter is a sequence of 16 bytes. This transport parameter MUST NOT be sent by a client but MAY be sent by a server. A server that does not send this transport parameter cannot use stateless reset for the connection ID negotiated during the handshake.

  Related optional features: Stateless Reset.
- **max_udp_payload_size (`0x03`)**: The maximum UDP payload size parameter is an integer value that limits the size of UDP payloads that the endpoint is willing to receive. UDP datagrams with payloads larger than this limit are not likely to be processed by the receiver.

  The default for this parameter is the maximum permitted UDP payload of 65527. Values below 1200 are invalid.

This limit does act as an additional constraint on datagram size in the same way as the path MTU, but it is a property of the endpoint and not the path. It is expected that this is the space an endpoint dedicates to holding incoming packets.

- **initial_max_data (0x04)**: The initial maximum data parameter is an integer value that contains the initial value for the maximum amount of data that can be sent on the connection. This is equivalent to sending a MAX_DATA for the connection immediately after completing the handshake.

Related optional features: Connection Flow Control.

- **initial_max_stream_data_bidi_local (0x05)**: This parameter is an integer value specifying the initial flow control limit for locally initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that sends the transport parameter. In client transport parameters, this applies to streams with an identifier with the least significant two bits set to 0x00; in server transport parameters, this applies to streams with the least significant two bits set to 0x01.

Related optional features: Stream Flow Control.

- **initial_max_stream_data_bidi_remote (0x06)**: This parameter is an integer value specifying the initial flow control limit for peer-initiated bidirectional streams. This limit applies to newly created bidirectional streams opened by the endpoint that receives the transport parameter. In client transport parameters, this applies to streams with an identifier with the least significant two bits set to 0x01; in server transport parameters, this applies to streams with the least significant two bits set to 0x00.
- **initial_max_stream_data_uni (0x07)**: This parameter is an integer value specifying the initial flow control limit for unidirectional streams. This limit applies to newly created unidirectional streams opened by the endpoint that receives the transport parameter. In client transport parameters, this applies to streams with an identifier with the least significant two bits set to 0x03; in server transport parameters, this applies to streams with the least significant two bits set to 0x02.
- **initial_max_streams_bidi (0x08)**: The initial maximum bidirectional streams parameter is an integer value that contains the initial maximum number of bidirectional streams the endpoint that receives this transport parameter is permitted to initiate. If this parameter is absent or zero, the peer cannot open bidirectional streams until a MAX_STREAMS frame is sent. Setting this parameter is equivalent to sending a MAX_STREAMS of the corresponding type with the same value.
- **initial_max_streams_uni (0x09)**: The initial maximum unidirectional streams parameter is an integer value that contains the initial maximum number of unidirectional streams the endpoint that receives this transport parameter is permitted to initiate. If this parameter is absent or zero, the peer cannot open unidirectional streams until a MAX_STREAMS frame is sent. Setting this parameter is equivalent to sending a MAX_STREAMS of the corresponding type with the same value.
- **ack_delay_exponent (0x0a)**: The acknowledgment delay exponent is an

integer value indicating an exponent used to decode the ACK Delay field in the ACK frame. If this value is absent, a default value of 3 is assumed (indicating a multiplier of 8). Values above 20 are invalid.

- **max_ack_delay (`0x0b`)**: The maximum acknowledgment delay is an integer value indicating the maximum amount of time in milliseconds by which the endpoint will delay sending acknowledgments. This value SHOULD include the receiver's expected delays in alarms firing. For example, if a receiver sets a timer for 5ms and alarms commonly fire up to 1ms late, then it should send a max_ack_delay of 6ms. If this value is absent, a default of 25 milliseconds is assumed. Values of 2^{14} or greater are invalid.

- **disable_active_migration (`0x0c`)**: The disable active migration transport parameter is included if the endpoint does not support active connection migration on the address being used during the handshake. An endpoint that receives this transport parameter MUST NOT use a new local address when sending to the address that the peer used during the handshake. This transport parameter does not prohibit connection migration after a client has acted on a preferred_address transport parameter. This parameter is a zero-length value.

  Related optional feature: Connection Migration

- **preferred_address (`0x0d`)**: The server's preferred address is used to effect a change in server address at the end of the handshake. This transport parameter is only sent by a server. Servers MAY choose to only send a preferred address of one address family by sending an all-zero address and port (0.0.0.0:0 or [::]:0) for the other family. IP addresses are encoded in network byte order.

  Related optional feature: Server Preferred Address

  The preferred_address transport parameter contains an address and port for both IPv4 and IPv6. The four-byte IPv4 Address field is followed by the associated two-byte IPv4 Port field. This is followed by a 16-byte IPv6 Address field and two-byte IPv6 Port field. After address and port pairs, a Connection ID Length field describes the length of the following Connection ID field. Finally, a 16-byte Stateless Reset Token field includes the stateless reset token associated with the connection ID. The format of this transport parameter is shown in Figure 3 below.

  The Connection ID field and the Stateless Reset Token field contain an alternative connection ID that has a sequence number of 1. Having these values sent alongside the preferred address ensures that there will be at least one unused active connection ID when the client initiates migration to the preferred address.

  The Connection ID and Stateless Reset Token fields of a preferred address are identical in syntax and semantics to the corresponding fields of a `NEW_CONNECTION_ID` frame. A server that chooses a zero-length connection ID MUST NOT provide a preferred address. Similarly, a server MUST NOT include a zero-length connection ID in this transport parameter. A client MUST treat a violation of these requirements as a connection error of type TRANSPORT_PARAMETER_ERROR.

```
Preferred Address {
    IPv4 Address (32),
    IPv4 Port (16),
    IPv6 Address (128),
    IPv6 Port (16),
    Connection ID Length (8),
    Connection ID (..),
    Stateless Reset Token (128),
}
```

Figure 3: *Preferred Address Format*

- **active_connection_id_limit (0x0e)**: This is an integer value specifying the maximum number of connection IDs from the peer that an endpoint is willing to store. This value includes the connection ID received during the handshake, that received in the preferred_address transport parameter, and those received in NEW_CONNECTION_ID frames. The value of the active_connection_id_limit parameter MUST be at least 2. An endpoint that receives a value less than 2 MUST close the connection with an error of type TRANSPORT_PARAMETER_ERROR. If this transport parameter is absent, a default of 2 is assumed. If an endpoint issues a zero-length connection ID, it will never send a NEW_CONNECTION_ID frame and therefore ignores the active_connection_id_limit value received from its peer.

Related optional feature: Connection ID Issue & Exchange

- **initial_source_connection_id (0x0f)**: This is the value that the endpoint included in the Source Connection ID field of the first Initial packet it sends for the connection.
- **retry_source_connection_id (0x10)**: This is the value that the server included in the Source Connection ID field of a Retry packet. This transport parameter is only sent by a server.

If present, transport parameters that set initial per-stream flow control limits (initial_max_stream_data_bidi_local, initial_max_stream_data_bidi_remote, and initial_max_stream_data_uni) are equivalent to sending a MAX_STREAM_DATA frame on every stream of the corresponding type immediately after opening. If the transport parameter is absent, streams of that type start with a flow control limit of 0.

A client MUST NOT include any server-only transport parameter: original_destination_connection_id, preferred_address, retry_source_connection_id, or stateless_reset_token. A server MUST treat receipt of any of these transport parameters as a connection error of type TRANSPORT_PARAMETER_ERROR.

4 Authenticating Connection IDs

The choice each endpoint makes about connection IDs during the handshake is authenticated by including all values in transport parameters. This ensures that all connection IDs used for the handshake are also authenticated by the cryptographic handshake.

Only necessary if you implement the optional feature *Connection ID Issue & Exchange*.

Each endpoint includes the value of the Source Connection ID field from the first `Initial` packet it sent in the initial_source_connection_id transport parameter. A server includes the Destination Connection ID field from the first Initial packet it received from the client in the original_destination_connection_id transport parameter; if the server sent a Retry packet, this refers to the first Initial packet received before sending the Retry packet. If it sends a Retry packet, a server also includes the Source Connection ID field from the Retry packet in the retry_source_connection_id transport parameter.

The values provided by a peer for these transport parameters MUST match the values that an endpoint used in the Destination and Source Connection ID fields of `Initial` packets that it sent (and received, for servers). Endpoints MUST validate that received transport parameters match received connection ID values. Including connection ID values in transport parameters and verifying them ensures that an attacker cannot influence the choice of connection ID for a successful connection by injecting packets carrying attacker-chosen connection IDs during the handshake.

An endpoint MUST treat the absence of the initial_source_connection_id transport parameter from either endpoint or the absence of the original_destination_connection_id transport parameter from the server as a connection error of type `TRANSPORT_PARAMETER_ERROR`.

An endpoint MUST treat the following as a connection error of type `TRANSPORT_PARAMETER_ERROR` or `PROTOCOL_VIOLATION`:

- absence of the retry_source_connection_id transport parameter from the server after receiving a Retry packet.
- presence of the retry_source_connection_id transport parameter when no Retry packet was received.
- a mismatch between values received from a peer in these transport parameters and the value sent in the corresponding Destination or Source Connection ID fields of Initial packets.

If a zero-length connection ID is selected, the corresponding transport parameter is included with a zero-length value.

Figure 4 shows the connection IDs (with DCID=Destination Connection ID, SCID=Source Connection ID) that are used in a complete handshake. The exchange of Initial packets is shown, plus the later exchange of `1-RTT` packets that includes the connection ID established during the handshake.

Figure 5 shows a similar handshake that includes a Retry packet.

In both cases, the client sets the value of the initial_source_connection_id transport parameter to C1.

When the handshake does not include a Retry (Figure 4), the server sets original_destination_connection_id to S1 (note that this value is chosen by

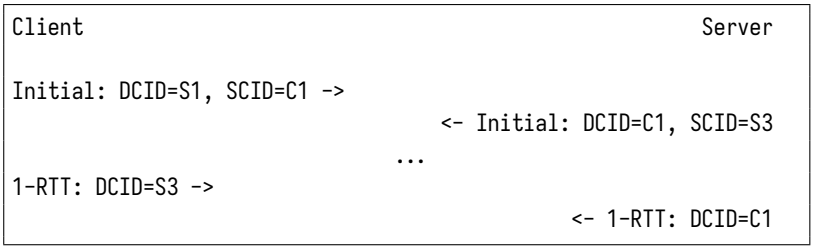

Figure 4: *Use of Connection IDs in a Handshake*

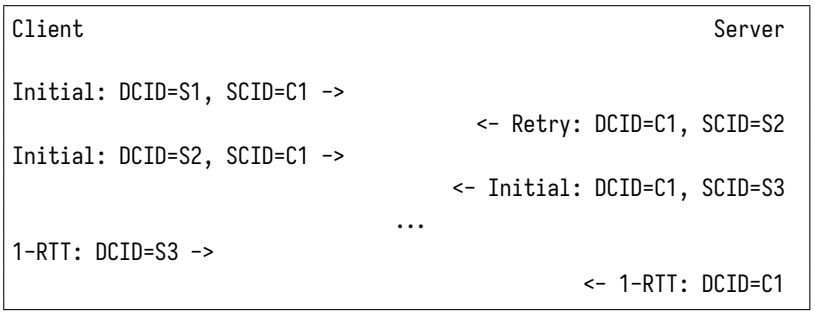

Figure 5: *Use of Connection IDs in a Handshake*

the client) and initial_source_connection_id to S3. In this case, the server does not include a retry_source_connection_id transport parameter.

When the handshake includes a Retry (Figure 5), the server sets original_destination_connection_id to S1, retry_source_connection_id to S2, and initial_source_connection_id to S3.